
Análisis comparativo entre microservicios y monolito modular aplicado al sistema distribuido TiendaTech

Gaibor Rodríguez Jeremy Ruperto

Resumen

Este informe compara la arquitectura de microservicios y el monolito modular, aplicados al sistema distribuido TiendaTech, PFC orientado al comercio electrónico de componentes de computadora y armado de equipos a pedido. El análisis considera modularidad, escalabilidad, despliegue, mantenibilidad, rendimiento y complejidad operativa. Los microservicios permiten dividir una aplicación en servicios autónomos, con despliegue y escalabilidad independiente, pero también exigen mayor control sobre comunicación, monitoreo, pruebas, seguridad y consistencia de datos. El monolito modular mantiene una sola unidad de despliegue, aunque organiza el sistema en módulos internos con responsabilidades claras. Para TiendaTech, donde se trabajan módulos como usuarios, catálogo, inventario, carrito, pedidos, pagos y reportes, el monolito modular resulta una alternativa adecuada para la primera versión del sistema. Esta decisión permite ordenar el dominio antes de considerar una separación física de servicios [1], [2].

Palabras clave: microservicios, monolito modular, arquitectura distribuida, escalabilidad, mantenibilidad, TiendaTech.

Índice

1. Introducción	3
2. Metodología	3
3. Fundamentos teóricos	4
3.1. Arquitectura de microservicios	4
3.2. Arquitectura de monolito modular	4
4. Comparación técnica entre microservicios y monolito modular	5
4.1. Modularidad y separación de responsabilidades	5
4.2. Escalabilidad	5
4.3. Despliegue y operación	6
4.4. Mantenibilidad y evolución	6
4.5. Rendimiento y comunicación	7
5. Tabla comparativa	7
6. Figura de arquitectura propuesta	8
7. Modelo formal de decisión arquitectónica	9
8. Análisis crítico propio	10
9. Aplicación al PFC TiendaTech	11
10. Conclusiones	11

1. Introducción

Las arquitecturas distribuidas ayudan a organizar sistemas de software mediante componentes que se comunican entre sí y reparten responsabilidades. Dentro de este enfoque, los microservicios se han estudiado como un estilo que divide una aplicación en servicios pequeños, autónomos y relacionados con capacidades de negocio. Esta división puede facilitar la entrega continua y la evolución independiente, pero también obliga a manejar comunicación remota, pruebas, monitoreo, operación y coordinación entre servicios [3], [4].

El monolito modular aparece como una opción intermedia entre el monolito tradicional y los microservicios. No se trata de una aplicación desordenada, sino de un sistema desplegado como una sola unidad, pero organizado internamente en módulos con responsabilidades definidas. Esta forma de diseño conserva la simplicidad de una aplicación única y, al mismo tiempo, mejora la mantenibilidad interna. Además, puede preparar el camino para una futura migración hacia microservicios si el sistema llega a necesitarlo [1], [2].

El problema no es decidir qué arquitectura es mejor en general, sino cuál conviene para un proyecto específico. En TiendaTech, adoptar microservicios desde el inicio podría agregar más complejidad de la necesaria, sobre todo si el equipo todavía está ajustando el dominio del negocio. La decisión debe considerar el tamaño del sistema, la capacidad del equipo, la carga esperada y la independencia real que necesitan sus módulos [3], [5].

El objetivo de este informe es comparar de forma crítica los microservicios y el monolito modular, usando TiendaTech como caso de aplicación. La comparación busca justificar una decisión arquitectónica coherente con el tamaño del proyecto, la capacidad del equipo y las necesidades reales del sistema [1], [3].

2. Metodología

La metodología corresponde a una revisión bibliográfica comparativa aplicada al caso TiendaTech. Primero se seleccionaron fuentes académicas recientes sobre microservicios, monolito modular, migración arquitectónica, granularidad, rendimiento y prácticas DevOps. Se priorizaron publicaciones de ACM, IEEE, Wiley, Applied Sciences y Journal of Systems and Software, porque el trabajo requiere fuentes recientes, verificables y relacionadas con el tema [1], [4].

Después se definieron criterios de comparación: modularidad, acoplamiento, escalabilidad, despliegue, mantenibilidad, rendimiento, complejidad operativa y aplicación al PFC. Estos criterios permiten evitar una comparación superficial de ventajas y desventajas. Además, están relacionados con problemas tratados en la literatura, co-

mo granularidad, operación, automatización, comunicación entre servicios y costos de migración [3], [5].

Finalmente, los criterios se aplicaron a TiendaTech, tomando como referencia módulos propios de un comercio electrónico: usuarios, productos, inventario, carrito, pedidos, pagos y reportes. Con esto se busca identificar qué partes podrían mantenerse en un monolito modular y cuáles podrían extraerse como microservicios en una etapa posterior. La intención no es solo describir arquitecturas, sino justificar una decisión coherente con el estado actual del PFC [1], [2].

3. Fundamentos teóricos

3.1. Arquitectura de microservicios

La arquitectura de microservicios organiza una aplicación como un conjunto de servicios pequeños, autónomos y desplegables de forma independiente. Cada servicio representa una capacidad del negocio y se comunica con otros mediante mecanismos como API REST, mensajería o eventos. En un comercio electrónico, esta división puede verse en servicios de catálogo, inventario, carrito, pedidos, pagos y usuarios, cada uno con su propio ciclo de vida [3], [4].

Una ventaja de los microservicios es que permiten escalar componentes concretos según su demanda. Si el catálogo recibe muchas más consultas que los reportes, se puede escalar solo el servicio de productos en lugar de replicar toda la aplicación. Esta ventaja depende de una buena granularidad: servicios muy grandes reducen autonomía, mientras que servicios muy pequeños aumentan la comunicación y el trabajo operativo [3], [5].

Esta arquitectura también trae retos que no aparecen con la misma intensidad en una aplicación única. Al distribuir el sistema, se deben manejar descubrimiento de servicios, fallos de red, monitoreo, pruebas distribuidas, seguridad y consistencia de datos. Por eso, los microservicios no solo requieren diseño de software, sino también prácticas DevOps, automatización, observabilidad y mecanismos de recuperación ante fallos [4], [6].

3.2. Arquitectura de monolito modular

El monolito modular es una arquitectura donde el sistema se despliega como una sola aplicación, pero el código se organiza en módulos con responsabilidades separadas. Esta estructura no equivale a un monolito tradicional sin control, porque busca definir límites internos entre áreas como usuarios, productos, inventario, carrito, pedidos o reportes. En la literatura reciente, se presenta como una alternativa práctica para mantener simplicidad operativa sin descuidar la organización interna [1], [2].

Su principal ventaja es la menor complejidad inicial. Al existir una sola unidad de despliegue, la configuración, las pruebas, el monitoreo básico y la conexión con la base de datos son más simples que en una arquitectura distribuida. Además, los módulos se comunican dentro del mismo proceso, lo que reduce latencia y evita varios problemas asociados a la red. Por eso, puede ser adecuado para proyectos iniciales, equipos pequeños o sistemas que aún no justifican una división física [1], [2].

El monolito modular también tiene límites. Aunque el código esté organizado, toda la aplicación se despliega junta, de modo que un cambio pequeño puede requerir publicar todo el sistema. Además, si el equipo no respeta los límites internos, el proyecto puede terminar convertido en un monolito acoplado. Para que esta arquitectura funcione, se necesitan paquetes bien separados, control de dependencias e interfaces claras entre módulos [1], [2].

4. Comparación técnica entre microservicios y monolito modular

4.1. Modularidad y separación de responsabilidades

Las dos arquitecturas buscan separar responsabilidades, pero lo hacen de distinta manera. En microservicios, la separación llega al nivel de servicios independientes, despliegue, comunicación y, muchas veces, datos. En el monolito modular, la separación ocurre dentro del código fuente, mediante módulos que siguen formando parte de una misma aplicación. La diferencia es importante: separar físicamente aumenta autonomía, pero también obliga a controlar contratos, errores remotos y coordinación entre servicios [1], [3].

La modularidad no depende solo de usar microservicios. Un sistema distribuido mal dividido puede quedar más acoplado que un monolito modular bien diseñado, especialmente si los servicios dependen demasiado entre sí. Para TiendaTech, sería más prudente definir primero límites lógicos entre productos, inventario, carrito y pedidos. Después, si esos límites se mantienen estables, algunos módulos podrían convertirse en servicios independientes [3], [5].

4.2. Escalabilidad

La escalabilidad suele ser el argumento más fuerte a favor de los microservicios. Esta arquitectura permite asignar más recursos a servicios específicos cuando su carga aumenta. En un comercio electrónico, el catálogo puede recibir muchas consultas, mientras que reportes puede usarse con menor frecuencia. Aun así, la escalabilidad depende de la granularidad: dividir servicios sin un criterio claro puede aumentar el costo de comunicación y reducir los beneficios esperados [3], [5].

El monolito modular escala como una unidad completa. Si productos necesita más capacidad, se replica toda la aplicación, aunque otros módulos no tengan la misma carga. Esta limitación puede aceptarse en una primera versión, porque simplifica la operación. En TiendaTech, separar catálogo o carrito tendría sentido cuando existan pruebas que muestren una demanda diferente entre módulos, no solo por seguir una tendencia arquitectónica [1], [5].

4.3. Despliegue y operación

En microservicios, cada servicio puede desplegarse por separado. Esto permite actualizar inventario sin publicar de nuevo usuarios, reportes o pagos. Esa independencia se relaciona con DevOps, integración continua, contenedores y automatización. Sin embargo, también exige herramientas y procesos para construir, probar, desplegar y monitorear varios servicios sin perder control del sistema [4], [6].

El monolito modular simplifica el despliegue porque existe una aplicación principal. Esto reduce artefactos, variables de entorno, imágenes Docker, puertos, balanceadores y configuraciones de red. Para TiendaTech, esta simplicidad ayuda a avanzar primero en productos, carrito, inventario y pedidos. Si se necesita demostrar distribución, sería más equilibrado separar uno o dos servicios con justificación técnica, no dividir todo el sistema desde el primer prototipo [1], [2].

4.4. Mantenibilidad y evolución

La mantenibilidad depende de límites claros, organización del código, pruebas y documentación. Los microservicios pueden mejorarla cuando cada servicio tiene una responsabilidad concreta y puede modificarse sin afectar al resto. Pero si la división es incorrecta, un cambio puede tocar varios servicios y aumentar la coordinación. Por eso, la transición hacia microservicios debe considerar granularidad y no solo separación tecnológica [3], [5].

El monolito modular favorece la mantenibilidad inicial porque permite trabajar con una sola base de código organizada. Esto ayuda al equipo a comprender el dominio antes de distribuirlo. Además, los estudios sobre transición y modularización hacia microservicios muestran que dividir un sistema implica decisiones de granularidad, esfuerzo de refactorización y posibles costos de rendimiento. En TiendaTech, lo razonable sería empezar con módulos claros y extraer servicios solo cuando existan razones verificables [2], [3].

4.5. Rendimiento y comunicación

El rendimiento cambia bastante entre ambos enfoques. En un monolito modular, los módulos se comunican dentro del mismo proceso, lo que evita latencia de red y simplifica transacciones. Esto ayuda en operaciones como registrar un pedido, validar inventario y calcular el total. En microservicios, esas acciones pueden requerir llamadas remotas o eventos, lo que añade costos de comunicación y coordinación [2], [6].

En microservicios, no basta con medir la velocidad de cada servicio por separado. Una operación puede volverse más lenta si depende de varias llamadas entre servicios. En TiendaTech, separar pedidos, inventario y pagos puede ser útil en una etapa avanzada, pero exige manejar fallos parciales, reintentos, consistencia eventual o mecanismos de compensación. Sin esos controles, la separación puede afectar el rendimiento y la confiabilidad [5], [6].

5. Tabla comparativa

La Tabla 1 resume las diferencias principales entre microservicios y monolito modular en el contexto de TiendaTech. La comparación no presenta una arquitectura como superior en todos los casos. Más bien, muestra que el monolito modular conviene cuando se busca simplicidad inicial y orden interno, mientras que los microservicios convienen cuando existen necesidades reales de escalabilidad independiente, despliegue autónomo y separación por capacidades de negocio [1], [3].

Cuadro 1: Comparación técnica entre microservicios y monolito modular aplicada a TiendaTech

Criterio	Monolito modular	Microservicios	Impacto en TiendaTech
Unidad de despliegue	Una sola aplicación	Varios servicios independientes	Conviene iniciar con un solo backend
Modularidad	Separación interna	Separación física	Primero se definen límites internos
Escalabilidad	Escala toda la aplicación	Escala servicios específicos	Catálogo o carrito podrían separarse después
Complejidad operativa	Menor complejidad inicial	Mayor operación y monitoreo	El equipo reduce carga técnica inicial
Rendimiento	Comunicación local	Comunicación remota	Menor latencia en la primera versión
Mantenibilidad	Depende del orden interno	Depende de límites correctos	Los módulos deben quedar bien separados
Base de datos	Puede iniciar centralizada	Puede requerir datos distribuidos	PostgreSQL centralizado es suficiente al inicio
Adecuación inicial	Alta para primera versión	Media sin alta demanda	La evolución debe basarse en uso real

6. Figura de arquitectura propuesta

La Figura 1 presenta el diagrama C4 nivel 3 del backend modular de TiendaTech. En esta vista, el backend se representa como un contenedor Spring Boot organizado internamente en componentes funcionales: usuarios, catálogo, inventario, carrito, pedidos, pagos y reportes. Esta estructura permite observar la separación lógica del sistema sin convertir todavía cada módulo en un servicio independiente.

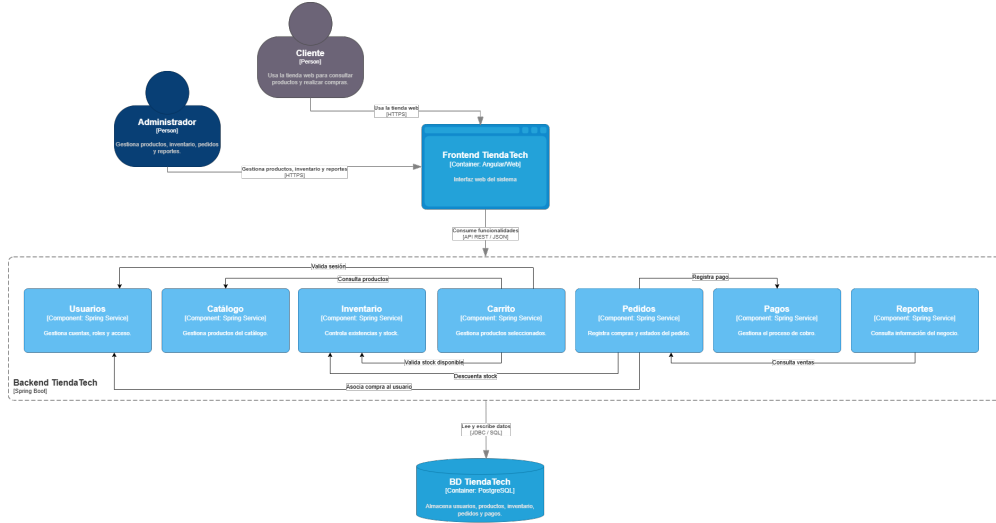


Figura 1: Diagrama C4 nivel 3 del backend modular de TiendaTech.

El diagrama muestra que TiendaTech puede organizar su backend mediante componentes internos bien definidos. Usuarios gestiona el acceso al sistema; catálogo administra los productos; inventario controla el stock; carrito maneja los productos seleccionados; pedidos registra las compras; pagos gestiona el cobro; y reportes consulta información del negocio. Esta organización reduce la complejidad inicial y facilita que el equipo trabaje sobre límites claros antes de pensar en una separación física de servicios [1], [2].

7. Modelo formal de decisión arquitectónica

A partir de los trade-offs de rendimiento identificados en la modularización hacia microservicios y de los desafíos operativos reportados en esta arquitectura, se propone un modelo simple de decisión para TiendaTech. El modelo no proviene directamente de las fuentes; es una síntesis propia para representar que una arquitectura distribuida puede aumentar el costo total cuando no existe una necesidad clara de independencia. Las fuentes respaldan los factores considerados, mientras que la formulación corresponde al análisis del autor [2], [6].

$$CA = CO + CC + CD \quad (1)$$

donde CA es el costo arquitectónico estimado, CO la complejidad operativa, CC el costo de comunicación entre componentes y CD el costo de despliegue y mantenimiento.

En TiendaTech, un monolito modular tendría menor CO y menor CC al inicio, porque los módulos se comunican dentro de una misma aplicación. Una arquitectura de microservicios, en cambio, aumentaría los tres términos por la existencia de varios servicios, llamadas remotas, monitoreo distribuido y despliegues independientes. Los microservicios se justificarían si sus beneficios compensan esos costos adicionales [4], [6].

También puede representarse la latencia de una operación distribuida. Si un pedido requiere consultar productos, validar inventario y registrar pago, la latencia total sigue la expresión:

$$L_T = L_P + L_{R_1} + L_I + L_{R_2} + L_{PA} \quad (2)$$

donde L_T es la latencia total, L_P la latencia del servicio de pedidos, L_{R_1} y L_{R_2} las latencias de red entre servicios, L_I la latencia de inventario y L_{PA} la latencia de pagos. Esta expresión es una aproximación propia inspirada en los trade-offs descritos en la literatura [2], [5].

Este modelo ayuda a justificar que la decisión arquitectónica debe mirar la operación completa. En TiendaTech, separar todos los módulos sin una estrategia de comunicación y tolerancia a fallos podría afectar rendimiento y mantenibilidad. La arquitectura debería evolucionar cuando existan métricas o necesidades funcionales que respalden la separación [3], [6].

8. Análisis crítico propio

La literatura revisada muestra que los microservicios pueden mejorar la escalabilidad y la autonomía de despliegue, pero esos beneficios no aparecen automáticamente. Para que funcionen bien, los límites entre servicios deben estar claros y el equipo debe contar con prácticas de monitoreo, automatización, pruebas y manejo de fallos. En TiendaTech, el principal riesgo no está en usar microservicios, sino en usarlos antes de conocer bien las reglas del negocio y las dependencias entre módulos [3], [4].

La granularidad es uno de los puntos más delicados. Un servicio demasiado grande pierde autonomía, mientras que servicios demasiado pequeños aumentan la comunicación y la coordinación. En TiendaTech, separar catálogo, inventario, carrito, pedidos y pagos desde el primer prototipo podría dificultar una operación que todavía se está definiendo. Por eso, antes de distribuir el sistema, conviene comprobar si esos módulos realmente cambian, escalan o fallan de manera independiente [3], [5].

El monolito modular tampoco debe entenderse como una solución definitiva. Su valor está en permitir que el sistema crezca de forma ordenada durante las primeras etapas. Si el catálogo empieza a recibir muchas consultas, si inventario requiere mayor control transaccional o si pagos necesita más aislamiento por seguridad, esos módulos podrían convertirse en candidatos a microservicios. La diferencia es que esa separación partiría de una necesidad concreta, no de una decisión tomada antes de validar el sistema [1], [2].

Para el PFC, la opción más realista es una arquitectura evolutiva. TiendaTech puede iniciar como una aplicación backend modular, con paquetes y responsabilidades bien separadas. Con pruebas de carga, errores detectados o nuevas necesidades del sistema, se podría decidir qué parte conviene separar. De esta manera, el proyecto demuestra criterios de arquitectura distribuida sin sobredimensionarse desde el inicio [1], [4].

9. Aplicación al PFC TiendaTech

TiendaTech es el PFC del equipo y se plantea como un sistema de comercio electrónico para vender componentes de computadora y permitir el armado de equipos a pedido. Para su primera versión, el backend puede organizarse como una aplicación Spring Boot con módulos internos para usuarios, catálogo, inventario, carrito, pedidos, pagos y reportes. Esta decisión se ajusta al estado actual del proyecto, porque permite avanzar con las funciones principales sin administrar varios servicios independientes desde el inicio [1], [2].

La posible evolución hacia microservicios debe analizarse por módulo. Catálogo sería candidato si concentra muchas consultas; inventario, si requiere mayor control sobre el stock; pedidos, si necesita trazabilidad independiente; y pagos, si se busca más aislamiento por seguridad o manejo de errores. Usuarios y reportes pueden permanecer dentro del backend modular mientras no tengan una carga o criticidad que justifique separarlos [3], [6].

Con esta estrategia, TiendaTech no queda atado a una sola arquitectura. El proyecto inicia con una estructura simple para el equipo, pero deja definidos los límites que podrían facilitar una separación futura. La decisión no depende solo de la teoría, sino de cómo crezca el sistema, qué módulos reciban más carga y qué partes necesiten independencia real durante el desarrollo [3], [5].

10. Conclusiones

El análisis realizado permite concluir que microservicios y monolito modular responden a necesidades distintas. Los microservicios son útiles cuando un sistema requiere escalabilidad independiente, despliegues separados, alta disponibilidad y equipos capaces de operar varios servicios. Sin embargo, también aumentan la complejidad de comunicación, monitoreo, pruebas, seguridad y consistencia de datos [4], [6].

El monolito modular resulta adecuado para sistemas en etapa inicial, siempre que no se confunda con un monolito desordenado. Su ventaja está en mantener una sola unidad de despliegue con módulos internos bien definidos. En TiendaTech, esta opción permite organizar usuarios, catálogo, inventario, carrito, pedidos, pagos y reportes dentro de una misma aplicación, reduciendo complejidad sin descuidar la separación de

responsabilidades [1], [2].

El enfoque más conveniente para TiendaTech es mantener una arquitectura inicial sencilla con límites internos bien definidos. Esta base permite desarrollar el sistema sin sobrecargar al equipo con operación distribuida prematura y, al mismo tiempo, deja abierta la posibilidad de separar módulos como catálogo, inventario, pedidos o pagos si el uso real del sistema lo exige [3], [5].

El código fuente y la documentación del proyecto están disponibles en el siguiente enlace: [Repositorio en GitHub](#).

Referencias

- [1] R. Su y X. Li, “Modular Monolith: Is This the Trend in Software Architecture?”, en *Proceedings of the 28th International Conference on Evaluation and Assessment in Software Engineering*, ACM, 2024, págs. 275-284. DOI: [10.1145/3643657.3643911](#).
- [2] N. Gonçalves, D. Faustino, A. R. Silva y M. Portela, “Monolith Modularization Towards Microservices: Refactoring and Performance Trade-offs”, en *2021 IEEE 18th International Conference on Software Architecture Companion (ICSA-C)*, IEEE, 2021, págs. 54-61. DOI: [10.1109/ICSA-C52384.2021.00015](#).
- [3] S. Hassan, R. Bahsoon y R. Kazman, “Microservice transition and its granularity problem: A systematic mapping study”, *Software: Practice and Experience*, vol. 50, n.º 9, págs. 1651-1681, 2020. DOI: [10.1002/spe.2869](#).
- [4] M. Waseem, P. Liang y M. Shahin, “A systematic mapping study on microservices architecture in DevOps”, *Journal of Systems and Software*, vol. 170, pág. 110798, 2020. DOI: [10.1016/j.jss.2020.110798](#).
- [5] S. Hassan y R. Bahsoon, “Systematic scalability analysis for microservices granularity adaptation design decisions”, *Software: Practice and Experience*, vol. 52, n.º 6, págs. 1432-1458, 2022. DOI: [10.1002/spe.3069](#).
- [6] M. Söylemez, B. Tekinerdogan y A. Kolukisa Tarhan, “Challenges and Solution Directions of Microservice Architectures: A Systematic Literature Review”, *Applied Sciences*, vol. 12, n.º 11, pág. 5507, 2022. DOI: [10.3390/app12115507](#).

Declaración de uso de IA: Para la estructuración inicial y revisión de redacción del presente informe se utilizó asistencia de inteligencia artificial generativa. Las ideas, argumentos, análisis crítico, aplicación al PFC y decisiones técnicas corresponden al autor.